R2771R1
Towards memory safety in C++

Draft Proposal, 2023-05-17

Author:
Thomas Neumann (TUM)

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

URL:
https://wg21.link/R2771/1

Source:
https://github.com/neumannt/memorysafety/blob/master/paper/memorysafety.bs

Date:
2022-05-23

Abstract

The lack of memory safety is one of the most concerning limitations of C++. This paper discusses a mechanism to make C++ memory safe while still keeping the traditional C++ programming concepts. By keeping track of dependencies between objects we can guarantee that an object will not be used after its dependencies have been destroyed or invalidated, which gives us temporal memory safety.

Table of Contents

§ 1. Revision History

R1 more precise dependency declarations, separate compile time from runtime checks more clearly, discuss interfaces

§ 2. Introduction

The lack of memory safety in C++ is a serious concern, as it is a common cause for security vulnerabilities, which has led the U.S. government to advise against using C++ for new projects. To stay relevant, C++ should evolve to guarantee memory safety by default (with explicit escape hatches to allow for low-level programming techniques in the few cases these are needed).

Historically C++ has already evolved to become safer, for example the introduction of smart pointers in C++11 eliminated large classes of memory bugs. But unfortunately it is still very easy to construct a dangling pointer when passing pointers around. In fact some new additions have made that even easier, for example constructing a (dangling) `string_view` from a temporary `string` is deceptively easy. That is very unfortunate, the language should be safe by default. Some programs will still need a mechanism for potentially unsafe operations, for example when directly interacting with hardware, but that should be an explicit opt-in, and limited to hopefully just a few places in the code that can then be carefully checked for bugs.

Sanitizer like, e.g., the address sanitizer supported by clang and gcc allow detecting many memory safety problems at runtime, but they are no panacea. Consider the example code below:

```
int main() {
   string s1 = "abcdefg";
   string_view s2 = s1;
   cout << s2.length() << " " << s2 << endl;
   s1.erase(2,2);
   cout << s2.length() << " " << s2 << endl;
}
```

None of the currently sanitizers report a problem here, but it is clearly an example of unsafe behavior, as the second print produces garbage output. In the case of `string` and `string_view` that is still relatively benign, as we only print nonsense in that case. But we could construct similar examples with `vector` and `span` where truly bad things would happen. The difficulty that the sanitizers have is that they do not understand the semantics of the operations. The underlying memory itself is still valid, but the contained objects have changed.

There have been multiple proposals to improve the situation. The most radical one is to switch to another language, at least for new code, in particular to Rust. Rust is famous for offering memory safety while still supporting low-level code like C++. Rust achieves this using its borrow checker, which guarantees that 1) an object is not destroyed (or moved) while a reference to it exists, 2) there is at most one mutable reference to an object, and 3) an object is not modified while a reference to it exists, except potentially through the single mutable reference. This leads to code that is both safe and efficient, but it requires a programming model that is quite different from what is commonly written in C++. And in general it is difficult to migrate an existing code base to Rust. Rust is a great language with many nice features, but it is not well suited for piece wise replacement of C++ code. For that to be an option there had to be a more or less seamless interaction between Rust and C++, but that is not the case. The lack of support for inheritance in Rust makes accessing C++ objects problematic, and the programming patterns of Rust often look alien in C++.

Less radical approaches suggest bringing lifetime annotations to C++, which would allow the compiler to detect frequent causes of bugs. That is a good idea, but not sufficient for the general case. We want that the compiler guarantees memory safety, having a linter that catches common problems is not good enough. Note that we will frequently only be able to guarantee memory safety in new code that helps the compiler with proving that the code is safe. But that is okay as long as that code can be introduced piece by piece, since there must be seamless interaction between the large existing code and new code.

In particular, it is not an option to simply introduce the Rust borrow checker in C++. It is fundamentally incompatible with common C++ idioms like, e.g., iterators. Enforcing the Rust rules would require a very different program style, and would not naturally interact with existing code. Instead, we will introduce a different set of rules that guarantees memory safety, too, but that is compatible with how current C++ code looks like. Sometimes the compiler will need annotations from the programmer to allow for reasoning, but that should mainly affect library writers. Simple code should just work. And existing code can seamlessly interact with the new code. It will not magically get the safety guarantees, of course, but the code will work correctly, and code can migrate piece wise to the memory safe model.

Note that the goal of this paper is mainly to show that it is possible to make C++ memory safe, and to discuss the challenges in doing so. We take a first stab at a mechanism that can be used to reach that goal, fully aware that there might be others that might work even better. Also, the syntax used here is pretty ad-hoc and not carefully designed, this would require more work if the committee considers this approach interesting. But what we want to emphasize is that full memory safety is possible, and we should not aim for less when changing the language.

Note further that this paper focuses on temporal memory safety, i.e., preventing access to destroyed objects and preventing access to invalid object states. Spatial memory safety is required, too, but that can be guaranteed with conceptually simpler mechanisms like, e.g., preventing raw pointer arithmetic in safe code, using `span` with mandatory bounds checks instead, etc. We also have to forbid `reinterpret_cast`, down-cast with `static_cast` (`dynamic_cast` is allowed), `const_cast`, `mutable`, plain `union` (`variant` is fine), and uninitialized pointers, as these break the type system. These mechanisms will still be available in regions that are explicitly marked as unsafe (and that are hopefully carefully checked for bugs).

The approach presented here guarantees temporal memory safety by keeping track of dependencies. When an object A (e.g., a `reference_wrapper`) depends on an object B, we must not use A after B has been destroyed. We consider that a dependency on the existence of an object. Similar, when an object A (e.g., a `string_view` or an iterator) depends on the content of an object B, we must not use A after B has been modified. We consider that a dependency on the content of an object. When we enforce both types of dependencies in the program we can guarantee temporal memory safety.

We already have a proof-of-concept implementation that guarantees memory safety using dependency tracking [ms], but without compiler support the dependencies have to be annotated and checked manually, which is undesirable. In the following we will therefore introduce mechanisms to allow the compiler to reason about dependencies and reference aliases, which is essential to enforce the invariants described above automatically.

§ 3. Concepts

We assume that for a long time there will be mixture of memory safe and non memory safe code. Migrating an existing code base takes a long time, and it has to be possible to do that incrementally. For simplicity, we assume that there is some kind of opt-in mechanism like this mock syntax:

```
namespace oldcode {
   // traditional C++ code
}
namespace newcode {
   [[memorysafety]];
   // everything defined in that scope enforces memory safety
}
namespace newcode {
   // not migrated yet, will not check memory safety, but can
   // call everything from oldcode and newcode
}
```

In particular, the whole memory safety mechanism is opt-in in the sense that we get the same memory layout and (at least conceptually) generate the same machine code with and without memorysafety annotations, which is mandatory for piece wise migration. In particular, in the example `oldcode` can call `newcode` without restrictions.

Within the `memorysafety` scope the compiler guarantees memory safety as long as we call only code that is also marked with `memorysafety`. When calling code that is not marked as memory safe all checks are disabled and the result is assumed to be globally valid. To help with migration a compiler could emit a warning for such a call.

Within a memory safety region the compiler guarantees that no objects are used whose dependencies have been invalidated. We distinguish two kinds of dependencies:

1. a dependency on the __existance__ of an object. Typical example are pointers, we must not dereference a pointer that points to an destroyed object

2. a dependency on the __content__ of an object. Typical examples are `string_view` or iterators, we must not use a string view after then underlying string has been been modified (or destroyed).

An object whose dependencies have been invalidated must not be used except for trivial destructors. Note that this is weaker than the Rust borrow checker requirement, which mandates that no reference to a destroyed object exists. But requiring that would be incompatible with C++ iterators, as seen below:

```
void foo(vector<int>& a) {
   if (!a.empty()) {
      auto i = a.begin();
      if ((*i) < 5) {
         a.push_back(3);
         // i is invalid now,
         // but that is okay as long as i
         // is not dereferenced
      }
   }
}
```

That is non-trivial to guarantee in general, thus we now discuss the mechanisms to allow the compiler to reason about dependencies. We distinguish compile time checks and runtime checks. We need both, unfortunately, for full memory safety, but common programming mistakes can already be caught with pure compile time checks. We assume a compiler will offer to switch between full memory safety and compile time only checks for systems where the runtime overhead is unacceptable.

§ 4. Compile Time Checks

that we need for the compiler to deduce safety. We start with the problem of aliasing. Note that in the following we will use the term reference to mean both pointers and references. To simplify the discussion we also pretend as if every modification of or access to of an object member happens through a corresponding access method. This conceptual method could always be inlined, thus it is not a real restriction in practice.

§ 4.1. Aliasing

In order to reason about safety and the lifetime of objects, the compiler must be able to detect if two references access the same object. In current C++ code that is not the case in general. Consider the

following example

```
void foo(vector<int>& a, const vector<int>& b) {
   if (!b.empty()) {
      a.push_back(b.front());
      // Unsafe, can crash when calling foo(x, x)
   }
}
```

The code looks innocent, but it is actually unsafe when `a` and `b` are aliasing each other. We could add a check for that to `foo`, but we want that code is safe by default. Thus, in memory safe code, we introduce an aliasing requirement: **When passing a non-const reference to a function or method, no other reference may alias that non-const reference**. Violations of that requirements are rejected by the compiler. Note that the implicit `this` argument is a reference, too.

Consider these examples, calling the function `foo` from above:

```
void bar(int x, int y) {
   vector<int> a, b;
   array<vector<int>, 2> c;
   foo(a,b); // ok
   foo(a,a); // error
   foo(c[x], c[y]); // error, cannot prove that they do not alias

   auto& r1 = &c[x];
   auto& r2 = &c[y];
   if (&r1 != &r2)
      foo(r1, r2); // safe now
}
```

Alias analysis is essential for the rest of the memory safety proposal, as otherwise the compiler has no chance to detect unsafe operations. However, we sometimes do want to allow aliasing, but that has to be announced to the compiler:

```
void swap(auto& a, [[mayalias(a)]] auto& b) { ... }
```

Now we can safely call `swap` with potentially aliasing arguments and the compiler will make sure that `swap` is safe even if `a` and `b` are the same object.

Note that we made aliasing opt-in instead of opt-out as otherwise too much code breaks (a lot of code is unsafe when arguments alias). And we explicitly list which arguments might alias each other, which allows for more fine-grained reasoning.

Note that the aliasing mechanism only tackles the problem of direct aliases. There are also more subtle indirect aliasing (or rather: dependency) problems, as illustrated below, but these are caught by the dependency tracking we discuss next:

```
void foo(vector<string>& a, const string& b) {
   a.push_back(b);
}
void bar(vector<string>& a) {
   // no direct alias, but nevertheless bad. Will be caught by dependencies
   foo(a, a.front());
}
```

§ 4.2. Dependency Tracking

Conceptually, we have to enforce a simple rule: A reference to an object must not be dereferenced once the object has been destroyed. When we cannot be sure about all uses, for example because the reference is stored outside a local variable, the reference must not exist longer than the underlying object. This forces us to keep track of lifetimes.

Lifetimes are modeled as dependencies. A global object has an infinite lifetime, i.e., every object can depend on a global object. (For now we ignore the problem of global constructor/destructor order, this might require more thoughts in the future). A local object has a lifetime defined by its scope. When a local object is destroyed, all objects that depend upon that object must a) have trivial destructors, and b) no methods must be called on the depending object. Consider the example below:

```
void foo() {
   int* a;
   {
      int d = 2;
      a = &d; // a depends on d
      *a = 3; // ok, d is alive
   }
   // ok so far even though we have a dangling pointer
```

```
   // would error: *a = 5; d is gone
}
```

When passing data from outside to a function we sometimes need explicit lifetime annotations. If none are given, we require that the lifetime of no reference argument may depend on the lifetime of a non-const reference argument (or non-const global). Consider the example below:

```
void bar(int* x),
void foo(int& x, int*& y) {
   int* a = &x; // ok, a depends on x
   bar(a); // ok, x is alive

   // error: y might live longer than x
   *y = a;
}
```

Here, we have to forbid the assignment to `y` as we cannot guarantee that `y` will not be used after `x` has been destroyed. If we want to allow that, we have to annotate the function to propagate the dependency:

```
void foo(int& x, [[maycapture(&x)]] int*& y) {
   // ok, the caller can check the lifetimes of x and y
   *y = &x;
}
```

For class methods we add a similar annotation if we store something that depends on a reference argument, and we describe if the return value depends on function arguments:

```
[[dependson(x,y)]] char* foo(char* x, char* y) {
   return x < y ? x : y;
}
[[maycapture(x)]] void Foo::bar(int* x) {
   this->y = x;
}
```

These annotations allow us to detect when an object depends upon a destroyed object, but they are not sufficient to handle, e.g., the `string_view` example from the introduction. There, the underlying `string` still exists, but its state has changed. Similar for iterators, we usually have to invalidate

iterators when the underlying container has changed. This can be annotated by capturing the __content__ of an object like this:

```
[[dependson(*x)]] string_view foo(string& x) {
   return x;
}
```

This introduces a new rule: **When an object A depends on the __content__ of an object B, A must not be used after a non-const method of B has been invoked**. Note that a non-trivial destructor counts as use. This affects objects like `span`, `string_view`, and iterators that do not own the underlying object, but that have to be invalidated when the underlying object is modified.

Conceptually this is a simple rule that says that an object state is modified only by a non-const method, thus all objects that depend upon the state of an object become invalid once a non-const method is invoked. In practice there are some difficulties, though, as sometimes methods are intentionally non-const even though they do not modify the state of an object. The most prominent ones are `begin` and `end`, which do not modify the container itself, but that do provide non-const access to container elements. Clearly, we do not want to invalidate all iterators when `begin` or `end` are called. Thus, we mark these functions as non-mutating, i.e., keeping objects that depend on the content valid even though they are non-const. The compiler must enforce that these functions do not call other non-const functions (except those marked as nonmutating):

```
template <class T>
[[dependson(*this),nonmutating]] myvec<T>::iterator myvec<T>::begin() {
   return iterator(this->ptr);
}
```

The third dependency rule that we introduce is that **a function argument must not depend on another non-const function argument**. That rule allows us to detect the `a.push_back(a.front())` example mentioned above. The result of `a.front()` depends on the content of `a`, which prevents it from being passed as an argument to `a.push_back(...)`.

This formalism handles most use cases, but there is one kind of construct that is used in the C++ standard library that is forbidden by the rules we have seen so far, and that is intentional invalidation. Consider this example fragment with a call to `std::vector::insert`:

```
a.insert(a.end(), b.begin(), b.end());
```

Our dependency rules say that `b.begin()` and `b.end()` must not depend upon `a`, and for good reasons. But the first argument to `insert` is different, there we do not only accept a dependency but we actually require that the iterator belongs to the same container. In many cases we will not be able to prove that statically, and implementing `insert` itself is also tricky because the size change of the `vector` invalidates the position iterator. For now we accept that implementing `insert` requires unsafe code (and an assert to make sure that the iterator indeed references the vector) and just introduce an annotation to allow this dependency:

```
template<class InputIt>
[[dependson(*this)]] iterator insert([[maydependon(*this)]] const_iterator
```

§ 4.3. Annotating Types

To automate the compile time checks, the compiler has to recognize the dependencies that occur within a program. For that, we first need a notion of a dependency-carrying type, which we call a `reference` here. Regular pointers and references are implicitly considered a reference. User defined types must be annotated as reference carrying if they capture dependencies:

```
// We need an annotation here
class [[references]] Class1 {
   Foo* x;
   public:
   [[maycapture(nx)]] void set_x(Foo* nx) { x=nx; };
   ...
};
// No annotation is needed here, we do not capture
class Class2 {
   Foo* pimpl = new Foo();
   public:
   ...
};
```

With that, the compiler recognizes objects that potentially depend on other objects whose lifetime they do not control. This information is used for the three cases of capturing annotations:

```
// We make a copy of x
[[maycapture(x)]] void foo1(Foo& x);
// We keep the address of x
```

```
[[maycapture(&x)]] void foo2(Foo& x);
// We keep the address of x and the content of x must not change
// (e.g., because we store an iterator)
[[maycapture(*x)]] void foo3(Foo& x);
```

Note that the annotation of `foo1` is a no-op if `x` is not itself marked as reference, as capturing a non-reference cannot create new dependencies. Nevertheless we need these annotations for generic programming, as x might sometimes be a reference. Inversely, an object `a` is only allowed to capture an object `b` if `a` is marked as `references`, as this tells the compiler to keep track of dependencies.

§ 4.4. Defining Interfaces

Dependency annotations are challenging when defining generic interfaces, like virtual functions, function pointers or in generic programming. The first question is, should the dependency annotations be part of the type system? That is, is the type of `void foo(int** a, int& b)` different from `void bar([[maycapture(&b)]] int** a, int& b)`? From a safety perspective it makes sense to treat them differently, but if we allow overloading based upon dependency annotations that clashes with the rule that it is safe to ignore all C++ attributes. Thus, for now we do not consider them part of the type system when overloading, but the compiler should refuse to cast a function type into another unless the capture set is a super set of the original captures.

For virtual functions we have same behavior, it is permitted to capture less than the base class but not to capture more:

```
class [[references]] Base {
   [[maycapture(&x)]] virtual void foo(Foo& x, Bar& y) = 0;
};
class Derived1 : public Base {
   [[maycapture(&x)]] void foo(Foo& x, Bar& y) override; // ok, we capture
};
class Derived2 : public Base {
   void foo(Foo& x, Bar& y) override; // ok, we capture less
};
class Derived3 : public Base {
   [[maycapture(&y)]] void foo(Foo& x, Bar& y) override; // error, we captu
};
```

With generic programming we have the problem that we do not even know if the values that we are handling carry dependencies themselves. To cover this, we extend the `[[references]]` attribute

into a `[[references(A,B,C,...)]]` attribute to express that the type is referencing another if any of the `A,B,C,...` list is a reference. With that, we can provide an interface for a generic vector, which shows the different annotations that we need:

```
template <class T>
class [[references(T)]] my_vec {
   ...
   /// We copy x, which might or might not propagate a dependency
   [[maycapture(x)]] void push(const T& x);
   ...
   class [[references]] iterator { ... };
   ...
   [[dependson(*this),nonmutating]] iterator begin();
   ...
};
```

To summarize, the interface has to tell the compiler 1) what is a reference like type, and 2) when to we create or store a reference to what. No annotations are needed when an argument is simply used inside a function and no reference escapes the call, which will be the majority of function calls.

§ 5. Runtime Checks

For fully memory safety it is unfortunately unavoidable to perform some runtime checks, which we will discuss next. In contrast to the compile time checks they cause runtime overhead, and thus it has to be possible to disable them. Some users might only want to enable them in debug builds to find potential dependency violations, as that will already find most problems in practice.

The rules discussed so far are useful for catching many common temporal memory safety bugs, but they are unfortunately not sufficient to catch all of them at compile time. Perhaps somewhat surprisingly, the main challenge is not so much reasoning about lifetimes but aliasing. Consider this hypothetical code snippet below:

```
void f1() {
   A a;
   B b;
   C c;
   a.push_back(123);
   f2(a, b, c);
   f3(b);
```

```
    f4(c);
    f5(b);
}
void f2(A& a, [[maycapture(&a)]] B& b, [[maycapture(&a)]] C& c) {
    b.a=&a;
    c.a=&a;
}
void f3(B& b) {
    b.i = b.a->begin();
}
void f4(C& c) {
    c.a->clear();
}
void f5(B& b) {
    b.e = *b.i;
}
```

Considered in isolation, none of the functions violate any lifetime rules. `a` outlives `b` and `c`, `f2` is properly annotated as capturing, and each function is harmless in itself. But the combination clearly violates memory safety because `f4` invalidates the iterator constructed in `f3`, and the compiler has no chance to detect that, at least if the functions are defined in different compilation units, as `b` and `c` are separate objects. One could try to introduce very elaborate annotations to describe that behavior, but that does not seem to be practical and would cause huge problems for generic code.

But what we can do instead is to detect the problem at runtime. Conceptually a dependency is like a soft lock, the referenced object must not end its lifetime before the dependent object. If it does, we consider the dependent object invalid, any operation except a trivial destructor will trigger an assert (and terminate the program). Similar for dependencies on the content of an object, here a call to a non-const method will invalidate the dependent objects. In a hypothetical memory-safety-sanitizer compilation the effective code could thus look as follows (the sanitizer calls are here added outside the called methods instead of inside to keep the code size reasonable):

```
void f1() {
    A a;
    B b;
    C c;
    memorysafety::mark_modified(&a);
    a.push_back(123);
    f2(a, b, c);
    f3(b);
    f4(c);
```

```
    f5(b);
    memorysafety::mark_destroyed(&c);
    memorysafety::mark_destroyed(&b.i);
    memorysafety::mark_destroyed(&b);
    memorysafety::mark_destroyed(&a);
}
void f2(A& a, [[maycapture(&a)]] B& b, [[maycapture(&a)]] C& c) {
    memorysafety::add_dependency(&b, &a);
    b.a=&a;
    memorysafety::add_dependency(&c, &a);
    c.a=&a;
}
void f3(B& b) {
    memorysafety::validate(b.a);
    memorysafety::add_content_dependency(&b.i, b.a);
    b.i = b.a->begin();
}
void f4(C& c) {
    memorysafety::validate(c.a);
    memorysafety::mark_modified(c.a);
    c.a->clear();
}
void f5(B& b) {
    memorysafety::validate(&b.i);
    b.e = *b.i;
}
```

In that version the `validate` call in `f5` fails, correctly detecting that the content of `a` has changed.

A proof-of-concept implementation of this checking infrastructure is available online [ms]. Like a typical sanitizer it is implemented using associate data structures, the underlying objects are left untouched. The implementation is not too expensive, `validate` is in O(1), `mark_modified` and `mark_destroyed` are amortized in O(1), and the dependency functions are logarithmic in the number of dependencies of an object. Nevertheless, these checks do cause overhead, thus one would probably use them like a typical sanitizer, detecting dependency bugs in debug builds and disabling the checks in release builds.

The runtime checks can handle arbitrary complex aliasing situations, and therefore one could ask why we even need the aliasing restrictions discussed above. The reason for these restrictions is that we want to detect as many problems at compile time as possible, and aliasing prevents that. Runtime checks are a kludge, and we would like to eliminate them as much as we can. Rust avoids the aliasing problem by enforcing that there can only be one reference to an object at any point in time if the object

is currently mutable. But that rule is too restrictive for typical C++ programs. Thus, we accept aliasing to some degree. Ideally, the compiler sees all aliases, which allows for detecting dependency (and content dependency) violations at compile time. If we cannot guarantee that, the runtime checks handle the remaining cases.

§ 6. Multi Threading

Even though this paper concentrates on temporal memory safety in the single threaded case, we ultimately need a solution for multi-thread memory safety, too. The only objects that are safe to share between threads are const objects, atomics, objects that are explicitly marked as thread-safe and that guarantee correctness themselves, and compounds of the former categories. To simplify building thread-safe objects, the standard library should probably provide something like a `locked<T>` wrapper that combines the `T` with a mutex and that only allows access to the underlying object via a callback mechanism:

```
locked<vector<int>> foo;
...
foo.apply([](vector<int>& c) {
   // This is implicitly protected by a mutex
   c.push_back(1);
   // The reference to c must not be captured
});
```

When we disallow capturing the pointer to the protected object (which we can, by forbidding capturing dependencies on the object), we can make sure that the object will only be accessed under the mutex, guaranteeing thread safety for that object. When creating a new thread we must make sure that only thread-safe objects can be passed by reference to the next thread.

One problem we still have with that approach is that global objects are implicitly shared between threads. Insisting that all global objects are thread-safe is impractical, in particular it would make simple programs that rely upon global state for simplicity much more complex. It seems more promising to do something similar to what the programming language `D` does, where globals are thread-local by default. Making all non-constants globals implicitly thread-local, and insisting on the thread-safety properties mentioned above for objects that are marked as shared might be one way to introduce thread safety in C++.

§ 7. Open Questions

The approach sketched above guarantees temporal memory safety by keeping track of dependencies between objects, both concerning the existence of objects and the content of objects, and by enforcing that an object is no longer used after its dependencies became invalidated. Syntax questions aside, there are still some open questions concerning this approach. In particular the balance between compile time checks and runtime checks is important. Clearly, we want to detect as many problems as we can at compile time. And conversely, we want to eliminate runtime checks as far as we can. It should be possible to eliminate some checks when, e.g., we can prove that a reference never escapes and no alias is constructed and thus no dependency violation can occur. But it is not clear yet how to best identify these situations. And perhaps some additional annotations could help the compiler to better reason about safety.

Furthermore, we need to combine temporal memory safety with spatial memory safety. The proof-of-concept implementation in [ms] provides a string implementation that offers both temporal and spatial memory safety in its iterators, but it introduces a bounds check at every iterator dereference. In theory that is not always necessary. For example the following code is guaranteed to be safe:

```
string foo;
for (char c:foo) {
    ...
}
```

The dependency rules make sure that the iterators used by the for loop stay valid, thus we do not need any bounds checks. Manual iterator operations however could very well go beyond the `end()` iterator and thus need bounds checks. Can we somehow differentiate these cases at compile time?

And finally we require a better solution for global objects. The multi-threading problems were already discussed above, but the undefined initialization order is not acceptable either. When a global object `A` depends on a global object `B`, `B` must be initialized before `A`. Currently there is no universal mechanism to guarantee that, let alone a fully automatic mechanism provided by the compiler. This has to be addressed, too, to reach full memory safety.

§ References

§ Informative References

[MS]

Thomas Neumann. *Memory Safety Checks*. URL: https://github.com/neumannt/memorysafety